

K-NN回归实战：基于Boston房价数据的预测分析

基于R自带数据集的案例分析

2026-03-20

1 1 引言

k-近邻算法（k-Nearest Neighbors, K-NN）通常被认为是分类问题的经典解决方案，但其应用远不止于此。实际上，K-NN同样能够处理回归问题，即预测连续数值型目标变量。K-NN回归的基本思想与分类版本一致：对于一个新的样本点，在训练集中找到其k个最近邻，并用这些邻居的目标变量值进行预测。本文将使用R自带的Boston数据集，完整展示K-NN回归在实际问题中的应用。该数据集记录了美国波士顿地区506个街区的房屋

2 2 数据加载与探索

```
#  
library(tidyverse)  
library(kknn)      # K-NN  
library(caret)      #  
library(MASS)       # Boston  
  
# Boston  
data(Boston)  
  
# 6  
head(Boston) %>%  
  knitr::kable(  
    caption = "Boston 6 ",  
    booktabs = FALSE,  
    format = "latex",  
    align = "c"  
  ) %>%  
  kableExtra::kable_styling(latex_options = "scale_down") # kableExtra
```

表1 展示了Boston数据集的前6行。该数据集包含14个变量，其中medv为房屋中位数价格（千美元），是我们需要预测的目标变量。

3 3 数据探索性分析

```
#  
summary(Boston) %>%  
  knitr::kable(  
    #  
  )
```

Table 1: Boston数据集前6行

crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0.00632	18	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
0.02985	0	2.18	0	0.458	6.430	58.7	6.0622	3	222	18.7	394.12	5.21	28.7

Table 2: Boston数据集中各变量的描述性统计

crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
Min.: 0.00632	Min.: 0.00	Min.: 0.46	Min.: 0.00000	Min.: 0.3850	Min.: 3.561	Min.: 2.90	Min.: 1.130	Min.: 1.000	Min.: 187.0	Min.: 12.60	Min.: 0.32	Min.: 1.73	Min.: 5.00
1st Qu.: 0.08205	1st Qu.: 0.00	1st Qu.: 5.19	1st Qu.: 0.00000	1st Qu.: 0.4490	1st Qu.: 5.886	1st Qu.: 45.02	1st Qu.: 2.100	1st Qu.: 4.000	1st Qu.: 279.0	1st Qu.: 17.40	1st Qu.: 375.38	1st Qu.: 6.95	1st Qu.: 17.02
Median: 0.25551	Median: 0.00	Median: 9.69	Median: 0.00000	Median: 0.5380	Median: 6.208	Median: 77.50	Median: 3.207	Median: 5.000	Median: 330.0	Median: 19.05	Median: 391.44	Median: 11.36	Median: 21.20
Mean: 3.61352	Mean: 11.36	Mean: 11.14	Mean: 0.00917	Mean: 0.5547	Mean: 6.285	Mean: 68.37	Mean: 3.795	Mean: 9.549	Mean: 408.2	Mean: 18.46	Mean: 356.07	Mean: 12.05	Mean: 22.53
3rd Qu.: 3.67708	3rd Qu.: 12.50	3rd Qu.: 18.10	3rd Qu.: 0.00000	3rd Qu.: 0.6240	3rd Qu.: 6.623	3rd Qu.: 94.08	3rd Qu.: 5.188	3rd Qu.: 24.000	3rd Qu.: 666.0	3rd Qu.: 20.20	3rd Qu.: 396.23	3rd Qu.: 16.95	3rd Qu.: 25.00
Max.: 88.97620	Max.: 100.00	Max.: 27.74	Max.: 1.00000	Max.: 0.8710	Max.: 8.780	Max.: 100.00	Max.: 12.127	Max.: 24.000	Max.: 711.0	Max.: 22.00	Max.: 396.90	Max.: 37.97	Max.: 50.00

```
caption = "Boston",
booktabs = FALSE,
format = "latex"
) %>%
kableExtra::kable_styling(latex_options = "scale_down")
```

表2 给出了各变量的最小值、第一四分位数、中位数、均值、第三四分位数和最大值。

```
ggplot(Boston, aes(x = medv)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black") +
  labs(title = " ",
        x = " ",
        y = " ") +
  theme_minimal()
```

图1 显示了房屋中位数价格的分布，整体呈右偏形态，价格主要集中在15-25千美元区间，高价房屋较少。

```
pairs(Boston[, c("rm", "lstat", "ptratio", "medv")],
      main = " ")
```

图2 展示了房间数（rm）、低收入人口比例（lstat）、师生比（ptratio）与房屋价格（medv）之间的散点图矩阵。可以看出

4 4 数据预处理

```
set.seed(2024)
train_index <- createDataPartition(Boston$medv, p = 0.8, list = FALSE)
train_data <- Boston[train_index, ]
test_data <- Boston[-train_index, ]

cat(" ", nrow(train_data), "\n")
```

```
##      407
```

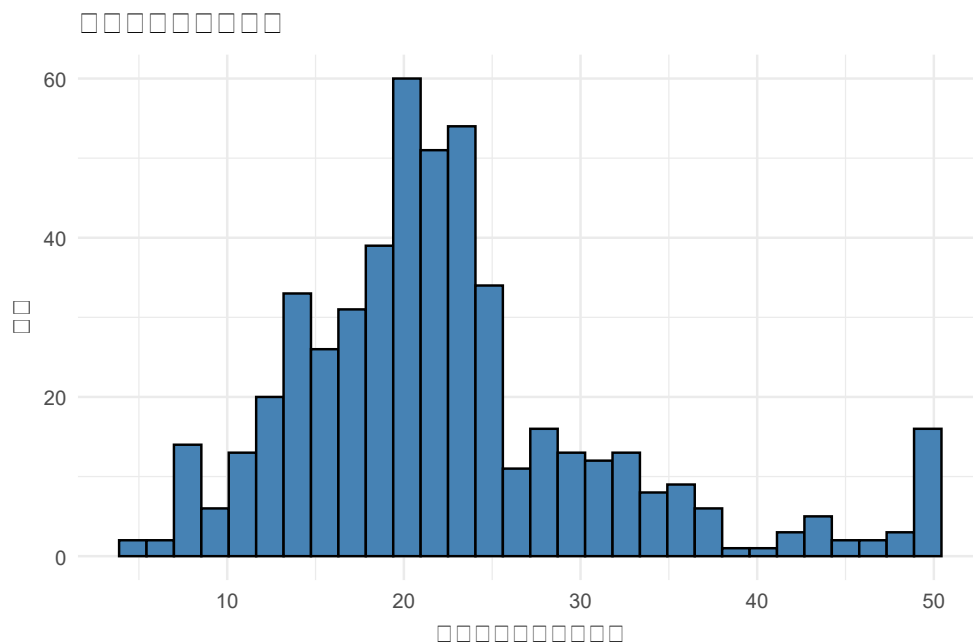


Figure 1: 房屋中位数价格分布

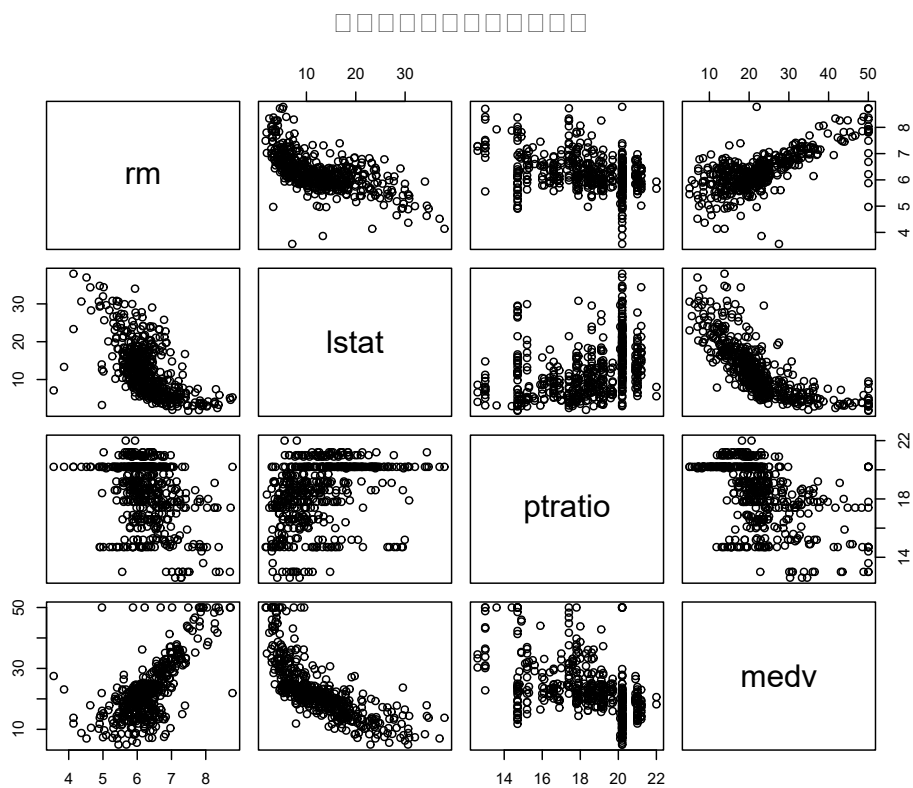


Figure 2: 关键特征与房屋价格的关系矩阵图

```
cat("      ", nrow(test_data), "\n")
```

```
##      99
```

```
#      Min-Max
preprocess_params <- preProcess(train_data[, -which(names(train_data) == "medv")],
                                method = "range")
train_scaled <- predict(preprocess_params, train_data)
test_scaled <- predict(preprocess_params, test_data)

train_scaled$medv <- train_data$medv
test_scaled$medv <- test_data$medv
```

5 5 K-NN回归建模

5.1 5.1 基础K-NN模型 (k=5)

```
knn_model <- kknnc(
  medv ~ .,
  train = train_scaled,
  test = test_scaled,
  k = 5,
  distance = 2,
  kernel = "rectangular"
)

test_scaled$pred_k5 <- fitted(knn_model)

rmse_k5 <- sqrt(mean((test_scaled$medv - test_scaled$pred_k5)^2))
mae_k5 <- mean(abs(test_scaled$medv - test_scaled$pred_k5))
r2_k5 <- cor(test_scaled$medv, test_scaled$pred_k5)^2

cat("      k=5 \n")
```

```
##      k=5
```

```
cat("RMSE ", round(rmse_k5, 2), " \n")
```

```
## RMSE  5.22
```

```
cat("MAE ", round(mae_k5, 2), " \n")
```

```
## MAE  3.14
```

```
cat("R2 ", round(r2_k5, 3), "\n")
```

```
## R2  0.696
```

当k=5时，模型在测试集上的R²为0.696，RMSE约为5.22千美元，作为基线表现尚可。

5.2 寻找最优k值

```
set.seed(123)
cv_results <- tibble(k = 1:20, rmse_cv = NA)

for (k_val in 1:20) {
  train_control <- trainControl(method = "cv", number = 5)
  knn_cv <- train(
    medv ~ .,
    data = train_scaled,
    method = "knn",
    trControl = train_control,
    tuneGrid = data.frame(k = k_val),
    preProcess = NULL
  )
  cv_results$rmse_cv[k_val] <- knn_cv$results$RMSE
}

ggplot(cv_results, aes(x = k, y = rmse_cv)) +
  geom_line(color = "steelblue", size = 1) +
  geom_point(color = "steelblue", size = 2) +
  geom_vline(xintercept = which.min(cv_results$rmse_cv),
    linetype = "dashed", color = "red") +
  labs(title = " k      RMSE",
    x = " k ",
    y = "RMSE ") +
  theme_minimal()
```

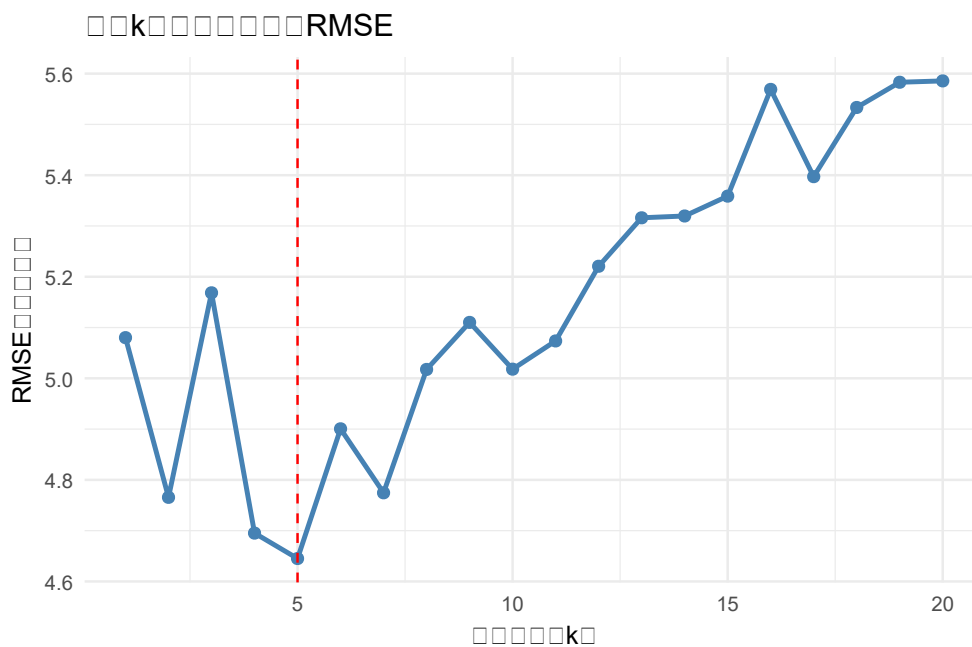


图3 展示了不同k值下的5折交叉验证RMSE。当k=5时，RMSE最小，因此选择该值作为最优k。

```

optimal_k <- which.min(cv_results$rmse_cv)
cat(" k ", optimal_k, "\n")

```

```
## k 5
```

```
cat(" k CV RMSE ", round(min(cv_results$rmse_cv), 2), " \n")
```

```
## k CV RMSE 4.65
```

5.3 使用最优k值重新建模

```

final_model <- kknnc(
  medv ~ .,
  train = train_scaled,
  test = test_scaled,
  k = optimal_k,
  distance = 2,
  kernel = "rectangular"
)

test_scaled$pred_optimal <- fitted(final_model)

rmse_optimal <- sqrt(mean((test_scaled$medv - test_scaled$pred_optimal)^2))
mae_optimal <- mean(abs(test_scaled$medv - test_scaled$pred_optimal))
r2_optimal <- cor(test_scaled$medv, test_scaled$pred_optimal)^2

cat("\n      k =", optimal_k, " \n")

```

```
##
##      k = 5
```

```
cat("RMSE ", round(rmse_optimal, 2), " \n")
```

```
## RMSE 5.22
```

```
cat("MAE ", round(mae_optimal, 2), " \n")
```

```
## MAE 3.14
```

```
cat("R2 ", round(r2_optimal, 3), "\n")
```

```
## R2 0.696
```

6 结果可视化

```
ggplot(test_scaled, aes(x = medv, y = pred_optimal)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed", size = 1) +
  labs(title = paste0("K-NN    k=", optimal_k, " "),
       x = "    ",
       y = "    ") +
  theme_minimal() +
  coord_equal()
```

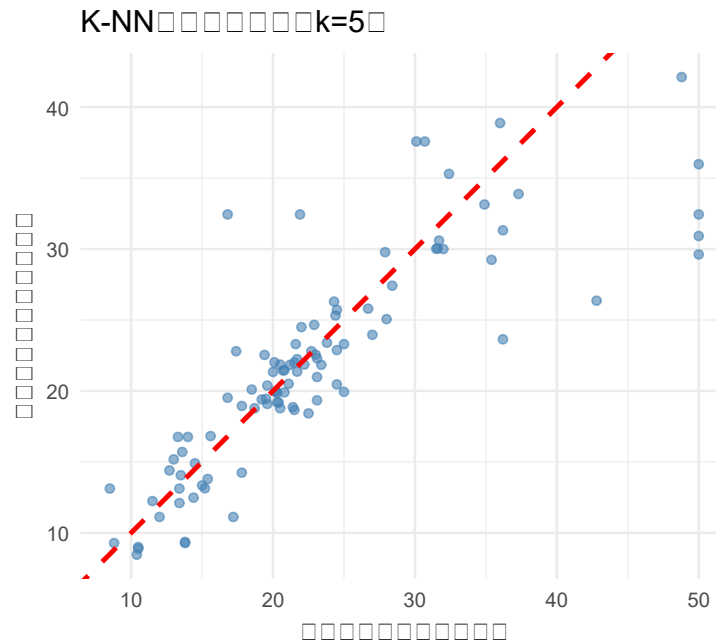


Figure 3: 最优K-NN模型的预测值与实际值对比图

图4 为测试集上实际值与预测值的散点图。如果模型完美，所有点应落在红色虚线上。可以看出大部分点围绕对角线分布，让

```
test_scaled$residuals <- test_scaled$medv - test_scaled$pred_optimal

ggplot(test_scaled, aes(x = pred_optimal, y = residuals)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed", size = 1) +
  geom_smooth(method = "loess", se = FALSE, color = "darkgreen") +
  labs(title = " ",
       x = "    ",
       y = "    ") +
  theme_minimal()
```

图5 展示了残差随预测值的变化。残差围绕零线随机分布，没有明显模式，说明模型没有系统性偏差。

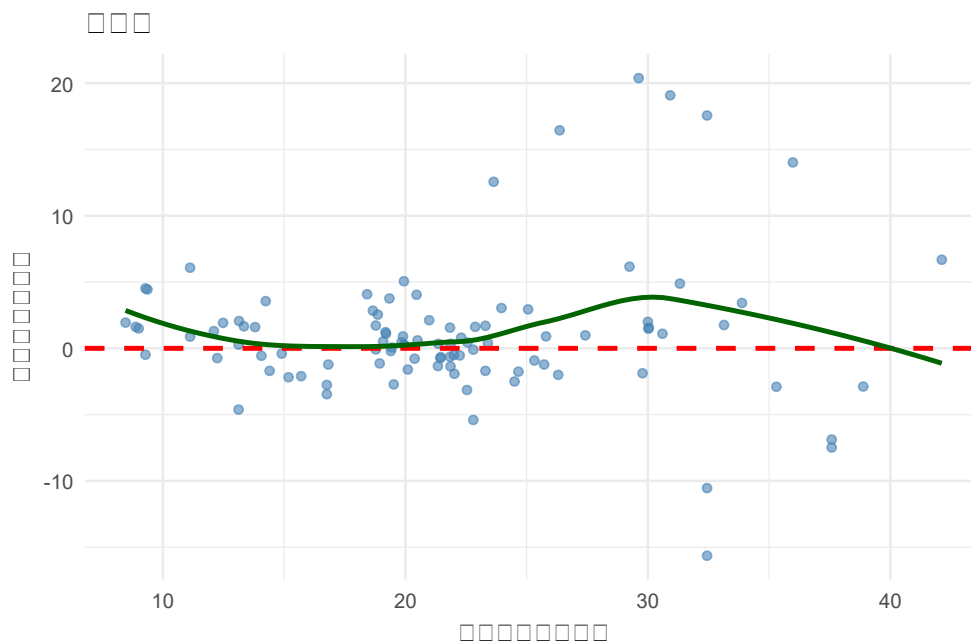


Figure 4: 残差分析图

7 7 加权K-NN回归（简单示例）

```
weighted_model <- kknn(
  medv ~ .,
  train = train_scaled,
  test = test_scaled,
  k = optimal_k,
  distance = 2,
  kernel = "gaussian"
)

test_scaled$pred_weighted <- fitted(weighted_model)

rmse_weighted <- sqrt(mean((test_scaled$medv - test_scaled$pred_weighted)^2))
mae_weighted <- mean(abs(test_scaled$medv - test_scaled$pred_weighted))
r2_weighted <- cor(test_scaled$medv, test_scaled$pred_weighted)^2

cat("\n K-NN      k =", optimal_k, " \n")
```

```
##
## K-NN      k = 5
```

```
cat("RMSE ", round(rmse_weighted, 2), " \n")
```

```
## RMSE 4.9
```

```
cat("MAE ", round(mae_weighted, 2), " \n")
```

```
## MAE 2.96
```

```
cat("R2 ", round(r2_weighted, 3), "\n")
```

```
## R2 0.733
```

采用高斯核（距离越近权重越大）的加权K-NN模型，其 R^2 为0.733，较等权重模型（ $R^2=0.696$ ）略有提升，说明考虑距离权

8 8 结果讨论

通过上述分析，我们可以得出以下结论：

1. K-NN回归的有效性：K-NN算法成功应用于Boston房价预测这一回归问题。最优模型的 R^2 达到0.696，表明所选特征包含
2. k值选择的影响：交叉验证结果显示，k值过小（如k=1）会导致过拟合；k值过大则可能导致欠拟合。通过系统性的k值
3. 特征尺度的重要性：在构建模型前对特征进行归一化是必要步骤。Boston数据集中不同特征的尺度差异很大（如tax数
4. 加权K-NN的优势：简单加权示例（高斯核）与等权重K-NN相比，RMSE和 R^2 有所改善，说明距离越近的邻居贡献越大的
5. 模型局限性：K-NN回归属于懒惰学习，预测时需要遍历整个训练集，计算开销较大。对于Boston这样506个样本的小数

9 9 总结

本文通过Boston房价预测案例，完整展示了K-NN回归在实际问题中的应用流程：从数据加载、探索性分析、预处理、建模、

NN算法不仅适用于分类问题，同样能够有效处理回归任务。

K-NN回归的优势在于其简单直观、无需训练过程、能够拟合复杂的非线性关系。然而，它也存在预测时计算开销大、对特征

NN回归可以作为基线模型，与线性回归、决策树、神经网络等方法进行比较，为问题选择最合适的建模方案。

10 10 参考文献

1. Mouselimis L. (2023). KernelKnn: Kernel k Nearest Neighbors. R package version 1.1.5.
2. Harrison, D. and Rubinfeld, D.L. (1978). Hedonic prices and the demand for clean air. Journal of Environmental Economics and Management, 5, 81-102.
3. Hechenbichler K. and Schliep K.P. (2004). Weighted k-Nearest-Neighbor Techniques and Ordinal Classification.
4. Kuhn M. (2008). Building Predictive Models in R Using the caret Package. Journal of Statistical Software, 28(5), 1-26. “ ‘